DSA0502 – Query Processing for Data Science	
Course Faculty	**Dr. B .SHYAMALA GOWRI**
Reg. No.	**192324164**
Student Name	**Sweety Roselin A**
Date	**30/07/2026**

CO2 AT2 – SCHEMA ANALYSIS AND DATA VOICE

System 1: University Course Registration System

> **Schema Tables:** Student(StudentID, Name, DepartmentID) | Department(DepartmentID, DepartmentName) | Course(CourseID, CourseName, FacultyID) | Faculty(FacultyID, FacultyName) | Enrollment(StudentID, CourseID, Semester)

Q1: Why is the Enrollment table necessary instead of storing CourseID directly in the Student table?

- **Many-to-Many Relationship:** Student and Course have an M:N relationship. A student takes multiple courses, and a course contains multiple students.
- **Normalization (1NF):** Placing CourseID directly in Student violates First Normal Form (1NF) by creating repeating columns or non-atomic comma-separated values.
- **Eliminates Redundancy & Anomalies:** Storing multiple rows per student duplicates demographic data (Name, DepartmentID), leading to update and insertion anomalies.
- **Junction Table Role:** Enrollment acts as an associative junction table using composite keys (StudentID, CourseID) to resolve the M:N relationship into two normalized 1:M relationships.

Q2: If the university wants to track grades for each enrolled course, how would you modify the schema?

- **Schema Modification:** Add a Grade column directly to Enrollment: Enrollment(StudentID, CourseID, Semester, Grade).
- **Functional Dependency:** Grade depends on both the student and the course taken in a specific semester. It is neither a property of Student alone nor Course alone.

System 2: Hospital Management System

> **Schema Tables:** Patient(PatientID, Name, Age) | Doctor(DoctorID, DoctorName, Specialization) | Appointment(AppointmentID, PatientID, DoctorID, Date) | Prescription(PrescriptionID, AppointmentID, Medicine)

Q1: What problems might occur if prescription details are stored directly in the Appointment table?

- **Cardinality Mismatch:** An appointment can yield multiple medicines, whereas Appointment represents a single consultation event.
- **Data Duplication:** Storing Medicine in Appointment forces duplication of PatientID, DoctorID, and Date for every medicine assigned.
- **Update & Insertion Anomalies:** Rescheduling requires updating multiple duplicated rows. Missing one causes data inconsistency.
- **1NF Violation:** Storing multiple medicines as comma-separated values violates atomic value constraints.

Q2: How would the schema change if a prescription contains multiple medicines?

- **Schema Redesign:** Split into a Master-Detail structure:
  1. **Prescription(PrescriptionID, AppointmentID, DateIssued)**
  2. **PrescriptionDetails(DetailID, PrescriptionID, Medicine, Dosage, Duration)**
- **Architectural Advantage:** Supports multi-item prescriptions without duplicating appointment metadata, achieving full 3NF compliance.

System 3: Online Shopping System

> **Schema Tables:** Customer(CustomerID, Name) | Product(ProductID, ProductName, Price) | Order(OrderID, CustomerID, OrderDate) | OrderDetails(OrderID, ProductID, Quantity)

Q1: Why is the OrderDetails table required in this schema?

- **Resolves M:N Cardinality:** Orders contain multiple products, and products appear in multiple orders.
- **Encapsulates Line-Item Attributes:** Stores attributes unique to specific order-product combinations (Quantity, UnitPriceAtPurchase).
- **Bridge Table Normalization:** Converts M:N complexity into two clean 1:M relationships (Order 1:M OrderDetails and Product 1:M OrderDetails).

Q2: What would happen if ProductID were stored directly in the Order table?

- **Single Item Limitation:** Restricts each order record to a single product.
- **Extreme Data Duplication:** Multi-product orders require replicating OrderID, CustomerID, and OrderDate across multiple rows.
- **Financial Inconsistencies:** Order totals, discounts, and tracking status require complex aggregation over redundant records, increasing error risks.

System 4: Library Management System

> **Schema Tables:** Member(MemberID, Name) | Book(BookID, Title, Author) | Issue(MemberID, BookID, IssueDate, ReturnDate)

Q1: Explain the relationship between Member and Book through the Issue table.

- **Temporal M:N Relationship:** A member borrows multiple books over time; a book is issued to multiple members over its lifespan.
- **Associative Entity Role:** Issue tracks individual borrowing events by pairing MemberID and BookID with operational dates (IssueDate, ReturnDate), preserving complete circulation history.

Q2: How would you modify the schema to track overdue fines?

- **Direct Enhancement:** Add FineAmount and FineStatus to Issue: Issue(IssueID, MemberID, BookID, IssueDate, DueDate, ReturnDate, FineAmount, FineStatus).
- **Dedicated Fine Ledger:** Create a separate Fine table: Fine(FineID, IssueID, FineAmount, PaymentStatus, PaidDate).
- **Architectural Rationale:** Fines are functionally tied to specific checkout events, keeping financial tracking isolated from member profiles.

System 5: Banking System

> **Schema Tables:** Customer(CustomerID, Name) | Account(AccountNo, CustomerID, Balance) | Transaction(TransactionID, AccountNo, Amount, Type)

Q1: Why should transactions be stored separately from account details?

- **Differing Data Lifecycles:** Account state (Balance) represents dynamic current snapshot data; Transaction records are immutable audit logs.
- **Unbounded Growth Prevention:** Prevents infinite growth of the Account table and eliminates repeated account metadata.

• **Auditability & Normalization:** Guarantees 3NF compliance and allows balance verification via historical transaction aggregation.

Q2: How would you redesign the schema to support joint accounts?

• **Schema Redesign:** Remove CustomerID from Account and introduce an AccountHolder bridge table:

1. **Customer(CustomerID, Name)**
2. **Account(AccountNo, Balance, AccountType)**
3. **AccountHolder(AccountNo, CustomerID, OwnershipType)**

• **Cardinality Shift:** Transforms the Customer-to-Account structure from 1:M to M:N, enabling joint accounts.

System 6: Employee Payroll System

> **Schema Tables:** Employee(EmployeeID, Name, DepartmentID) | Department(DepartmentID, DepartmentName) | Payroll(PayrollID, EmployeeID, Salary, Month)

Q1: Why is Department information separated into a different table?

• **3NF Normalization:** Department details (Name, Location) depend on DepartmentID, not EmployeeID.

• **Single Source of Truth:** Defines department data once, referenced by multiple employees via foreign keys without redundancy.

Q2: What redundancy issues would arise if department details were stored in Employee records?

• **Mass Duplication:** Repeats DepartmentName across all employee records in that department.

• **Update Anomalies:** Renaming a department requires updating hundreds of rows; missing one breaks data consistency.

• **Insertion Anomalies:** New departments cannot exist without assigning at least one employee.

• **Deletion Anomalies:** Deleting the last employee erases the department's existence.

System 7: Hotel Reservation System

> **Schema Tables:** Customer(CustomerID, Name) | Room(RoomID, RoomType) | Reservation(ReservationID, CustomerID, RoomID, CheckIn, CheckOut)

Q1: Why should reservation details be maintained separately from room details?

- **Static vs. Dynamic Separation:** Room holds static physical metadata; Reservation tracks dynamic temporal bookings.
- **Prevents Infinite Redundancy:** Avoids duplicating room properties for every historical and future booking.
- **Historical Logs:** Preserves guest stay history and revenue records without altering room availability.

Q2: How would you prevent double booking of rooms using the schema?

- **Overlap Condition:** Detect overlaps using: (NewCheckIn < ExistCheckOut) AND (NewCheckOut > ExistCheckIn).
- **Enforcement Strategies:** 1. Application Transaction Checks: Validate availability before inserting. 2. DB Triggers & Exclusion Constraints: Use BEFORE INSERT triggers or PostgreSQL temporal exclusion constraints (EXCLUDE USING gist) at the DB engine level.

System 8: Food Delivery Application

> **Schema Tables:** Customer(CustomerID, Name) | Restaurant(RestaurantID, Name) | Order(OrderID, CustomerID, RestaurantID) | DeliveryAgent(AgentID, Name) | Delivery(OrderID, AgentID)

Q1: Why is Delivery maintained as a separate table?

- **Decoupled Operational Lifecycle:** Order creation precedes agent assignment.
- **Logistics State Isolation:** Encapsulates fulfillment attributes (PickupTime, DeliveryStatus) separate from financial orders.
- **Agent Reassignment Flexibility:** Allows driver reassignment without modifying the order contract.

Q2: How would the schema change if multiple delivery agents could handle a single order?

- **Schema Redesign:** Convert 1:1 delivery mapping to an M:N bridge structure:
  DeliveryLeg(LegID, OrderID, AgentID, LegSequence, LegStatus)
- **Multi-Leg Support:** Enables multi-stage relay fulfillment, individual driver tracking, and leg-wise compensation.

System 9: Learning Management System (LMS)

> **Schema Tables:** Student(StudentID, Name) | Course(CourseID, CourseName) | Enrollment(StudentID, CourseID) | Assignment(AssignmentID, CourseID)

Q1: Why is Enrollment considered a bridge table?

- **Resolves M:N Cardinality:** Students take multiple courses, and courses have multiple students.
- **Structural Junction:** Links StudentID and CourseID, enabling 1:M relationships and storing enrollment metadata (EnrollmentDate, Status).

Q2: How would you modify the schema to record assignment grades?

- **Dedicated Submission Entity:** Create Submission(SubmissionID, StudentID, AssignmentID, Grade, SubmissionDate).
- **Architectural Rationale:** Grades belong to specific student-assignment submissions. They cannot reside in Course (1:M) or Enrollment (course-wide scope).

System 10: Bus Ticket Reservation System

> **Schema Tables:** Passenger(PassengerID, Name) | Bus(BusID, Route) | Booking(BookingID, PassengerID, BusID, TravelDate)

Q1: Why should booking details be stored separately from passenger information?

- **Entity Separation:** Passenger stores static personal identity; Booking records temporal journey transactions.
- **Storage Optimization:** Prevents repeating passenger demographics for every ticket booked.
- **Encapsulates Trip Data:** Isolates journey-specific attributes (SeatNumber, Fare, TravelDate).

Q2: If a booking contains multiple passengers, how would you redesign the schema?

- **Multi-Passenger Redesign:** Remove PassengerID from Booking and introduce a manifest bridge table:
  1. **Booking(BookingID, BusID, TravelDate, TotalFare)**
  2. **BookingPassenger(BookingID, PassengerID, SeatNumber)**
- **Architectural Impact:** Enables single-transaction group bookings linked to individual seat assignments.